CS-412 Software Security Fuzzing Lab
Fuzzing `libpng` with AFL++

Group 7
EPFL — Spring 2026

Abstract

We fuzzed `libpng` 1.2.56 with AFL++ using source instrumentation, QEMU mode, persistent mode, and focused API harnesses. The required 30-minute decoder baselines exercise `png_read_info()`, transform setup, `png_read_image()`, and `png_read_end()` with a CRC-neutralized libpng build and a small PNG seed corpus. We then extended the campaign suite to persistent decoding, QEMU, metadata, write, and text API targets. The strongest confirmed finding is a minimized reproduction of the CVE-2016-10087 `png_set_text_2` null-pointer dereference from the focused text-API harness; decoder hangs and non-reproducing crash files are tracked separately.

1 Setup and Methodology

Target and environment. The target is `libpng` 1.2.56. The Docker image builds AFL++, downloads the `v1.2.56` source, applies a CRC-neutralization patch, and installs three static library variants: AFL++ instrumentation with ASan, AFL++ instrumentation without ASan, and a vanilla `gcc` build for QEMU mode. The harnesses live in `src/`: `harness.c` is the main decoder, `harness_persistent.c` is the persistent-mode decoder, and `harness_text_api.c`, `harness_write_api.c`, and `harness_metadata_api.c` target focused libpng APIs.

Campaigns and artifacts. The required source-instrumented and QEMU decoder campaigns both ran for just over 32 minutes, giving the clean same-wall-clock evidence used in Q4 and Q7. The broader campaign suite was longer: persistent decoding, write APIs, and metadata APIs each ran for roughly six hours, and a longer QEMU decoder campaign ran for about 3 h 39 min. A separate focused text-API campaign produced the reproducible crash triaged in Q5. The compact evidence bundle under `triage-artifacts/` keeps `cmdline`, `fuzzer_stats`, `plot_data`, crashes, and hangs without the full AFL++ queues. Additional Q3 and Q8 evidence is stored under `evidence/`; crash-triage outputs are under `pocs/text_api_triage/`.

2 Evaluation Questions

Each subsection corresponds to one graded question.

2.1 Q1 — Harness Design

The primary fuzz target is the explicit read pipeline rather than a wrapper such as `png_read_png()`. AFL++ passes a testcase path through `@@`; the harness reads that file into an `InputState` buffer and installs a `png_set_read_fn()` callback. The callback copies exactly the number of bytes requested by libpng and raises `png_error()` on out-of-input reads, which models a normal stream while keeping execution deterministic.

After creating `png_struct` and `png_info`, the harness installs `setjmp(png_jmpbuf(...))` so malformed PNGs return cleanly instead of terminating the fuzzer. It then executes `png_read_info()`, applies `png_set_expand()`, `png_set_strip_16()`, `png_set_gray_to_rgb()`, background handling, and gamma correction, calls `png_read_update_info()`, allocates row buffers, reaches `png_read_image()`, and finishes with `png_read_end()`. This covers signature checks, chunk parsing, zlib inflation, filtering, row logic, and color conversion decisions.

The main resource guard rejects images wider or taller than 4096 pixels before row allocation. This does not validate PNG semantics; it prevents AFL++ from spending time or memory on huge allocations unlikely to improve bug-finding. Cleanup is centralized so success and libpng error paths free rows, destroy libpng state, close the file, and release the input buffer. The persistent harness preserves the same decode logic but wraps it in `__AFL_LOOP(1000)` to measure forkserver overhead separately in Q8.

We also added focused API harnesses because the file decoder does not naturally exercise all public setters. The text harness calls `png_set_text()` twice with fuzz-controlled text fields and optional `png_free_data()`. The metadata harness drives color, physical, profile, time, and unknown-chunk setters. The write harness creates synthetic rows and exercises `png_create_write_struct()`, output callbacks, row writing, filters, compression options, palette mode, bit depths, and transform flags.

2.2 Q2 — Instrumentation and Sanitizers

The source-instrumented build uses `afl-clang-fast` with `-fsanitize=address -g -O1`; the harness links statically against `/build/install/lib/libpng12.a` with `-fsanitize=address -lz -lm`. ASan turns heap overflows, use-after-free, and invalid accesses into deterministic crashes. We set `ASAN_OPTIONS` to abort on errors, disable leak detection, and skip inline symbolization; this makes sanitizer findings visible to AFL++ without leak noise.

For QEMU mode, the target is rebuilt with `gcc -g -O1`, no sanitizer, and no AFL++ compile-time counters, then run with `afl-fuzz -Q`. This simulates a binary-only target. For Q8, a third libpng variant uses `afl-clang-fast -g -O1` without ASan, allowing the same harness to separate sanitizer overhead from fork/persistent overhead.

The key source patch is `patches/libpng-1.2.56-no-crc.patch`, which changes `pngrutil.c`'s CRC check in `png_crc_finish()` to always report success. PNG chunk CRCs are invalidated almost immediately by byte-level mutation. Without the patch, libpng rejects critical chunks early or discards mutated ancillary chunks, starving deeper parsers of coverage feedback. With CRC neutralized, mutated chunk contents continue into chunk-specific code, decompression, transform setup, and metadata handling.

2.3 Q3 — Seed Corpus and Dictionary

The decoder corpus uses ten small valid PNGs, 3909 bytes total. They cover RGB, RGBA, grayscale, grayscale+alpha, 16-bit grayscale, indexed color with `PLTE`/`tRNS`, text metadata, Adam7 interlacing, and multi-`IDAT`/`hIST` cases. Small seeds keep mutation throughput high, while structural diversity starts AFL++ past the PNG signature and IHDR gate in several useful parser states.

Reader campaigns use AFL++'s built-in PNG dictionary, `/build/dictionaries/png.dict`, originally attributed to Michal Zalewski, with `-x`. In language terms, PNG is a binary grammar of the form *File* $\rightarrow$ *Signature Chunk*$^+$, where each chunk is `length / type / data / CRC`. Dictionary tokens such as the PNG signature and chunk-type terminals `IHDR`, `PLTE`, `IDAT`, `IEND`, `tRNS`, `gAMA`, and `tEXt` help AFL++ make mutations that preserve or create recognizable grammar terminals. Chunk ordering, lengths, CRCs, filter bytes, and compressed IDAT payloads remain semantic structure discovered through coverage feedback.

Because AFL++ does not write strategy-yield rows to `fuzzer_stats`, we captured two supplementary 300-second UI transcripts under `evidence/q3-live/`. The first kept deterministic stages enabled and filled the dictionary row; the second used `-z -u` to reach havoc and splice quickly. These are Q3 evidence runs, not replacements for the canonical Q4 campaign.

Strategy	Finds	Attempts	Yield
Dictionary overwrite	55	2,726	2.02%
Dictionary insert	0	2,754	0.00%
Havoc	61	6,500	0.94%
Splice	4	420	0.95%

Table 1: AFL++ strategy yields from live status captures.

Dictionary overwrite is high precision: it can flip a chunk type to a known terminal without changing file size, which explains its 2.02% yield. Dictionary insert is less aligned with PNG because inserted bytes often break chunk lengths. Havoc is lower precision but high volume, and splice is high variance: most combinations break global PNG consistency, but chunk-boundary recombinations can still expose new states.

2.4 Q4 — Campaign Analysis

The Q4 baseline is the required ASan source-instrumented decoder campaign on the ten PNG seeds with the PNG dictionary, target `./png_fuzz`. We use this 32-minute run because the lab asks for a 30+ minute instrumented campaign with AFL++ status counters and an `afl-plot` edge curve.

Metric	Value
Runtime	1,945 s (32.4 min)
Executions / speed	1,034,958 / 532.08 exec/s
Corpus count	715 files
Cycles done	5
Stability	100.00%
Bitmap coverage	30.94%
Edges found / total	966 / 3,122
Crashes / hangs	0 / 2

Table 2: Final AFL++ counters for the instrumented decoder campaign.

Stability is 100% and AFL++ recorded zero variable corpus entries, so the harness behaved deterministically. The queue grew from 10 seeds to 715 files, meaning AFL++ found many coverage-improving mutations through signature, IHDR, palette, interlace, and ancillary-chunk gates.

The edge curve has a fast-rise, slow-tail shape: 735 edges at 61 seconds, 876 at 325 seconds, and the last new edge at 1,677 seconds. The run then stayed flat at 966 edges for the final 268 seconds. That is local

saturation for this harness, seed set, dictionary, CRC-neutralized build, and ASan fork-mode budget. It is not full libpng saturation: 966 reached edges out of 3,122 instrumented edges leaves most code gated by semantic constraints, rarer transform combinations, error paths, and APIs not called by the decoder harness.

Longer supporting campaigns show what extra budget and alternate entry points added beyond the required baseline. The persistent decoder campaign reached 1,088 / 3,131 edges (34.75%) over roughly six hours, improving coverage but producing two saved crashes that did not reproduce as standalone ASan faults. The longer QEMU decoder run reached 2,917 bitmap entries in about 3 h 39 min. The write API campaign reached 558 / 2,090 edges with no crashes, and the clean metadata API campaign reached 458 / 3,388 edges with no crashes or hangs. These runs support the coverage story, while the confirmed bug comes from the focused text-API campaign triaged below.

2.5 Q5 — Crash Triage

The reportable crash comes from the focused text API campaign rather than the baseline decoder. We triage it because Q5 asks for one real crash found by our fuzzing setup, and the text harness exercises a public libpng setter state sequence that the file decoder does not naturally reach. We selected one AFL++ crash from `triage-artifacts/local/findings-api-cve-212823/main/crashes/` and replayed it with `./png_fuzz_api <input>`; the minimized reproducer, ASan trace, `afl-tmin` output, and deduplication logs are under `pocs/text_api_triage/`. The 39-byte original exited with signal 6 under ASan, and `afl-tmin` reduced it to the 3-byte input `00n` while preserving the crash, a 92.31% size reduction.

The minimized input produces an ASan null-page write in libpng. The top stack frames are `png_set_text_2()` in `pngset.c`, called by `png_set_text()` at `pngset.c:654`, then by our `target_text_api()` at `harness_text_api.c:112`. This reproduces the CVE-2016-10087 bug class: libpng versions through 1.2.56 have a null-pointer dereference in `png_set_text_2()` when an application adds, removes, and re-adds text chunks.[1] Our harness exercises that state sequence by calling `png_set_text()`, optionally freeing `PNG_FREE_TEXT`, then calling `png_set_text()` again with fuzz-controlled fields.

For deduplication, we replayed all 90 saved text API crash files against the same ASan harness. Of these, 59 reproduced as the same `png_set_text_2` ASan SEGV and 31 exited cleanly in standalone replay, so we count one confirmed root cause and treat the clean standalone replays as non-reproducing AFL artifacts. Decoder hangs remain pending because they still need timeout reproduction, minimization, and root-cause analysis. The persistent decoder's two saved crashes are excluded unless they can be reproduced as standalone ASan faults, and the early metadata API crashes are excluded because they were caused by a harness allocation bug and the clean rerun produced zero crashes.

2.6 Q6 — Attack Surface Analysis

Two realistic consumers of libpng are Firefox and ImageMagick. Firefox's PNG decoder lives in `image/decoders/nsPNGDecoder.cpp`: it includes `png.h`, creates libpng read state, configures unknown chunks and limits, and uses progressive callbacks through `png_set_progressive_read_fn()`.[2] ImageMagick's PNG coder lives in `coders/png.c`: it contains read and write entry points such as `ReadPNGImage()`, `ReadOnePNGImage()`, and `WritePNGImage()`, and calls libpng through application-specific blob I/O, options, profile handling, and MNG/JNG support.[3]

In a browser scenario, an attacker can place a malicious PNG in a web page, favicon, CSS image, downloaded attachment preview, or image cache path. The exposed surface is the complete browser image pipeline around progressive network input, metadata decode, color management, frame allocation, and surface output. In an ImageMagick scenario, a server may call `identify`, `convert`, or a library binding on user-uploaded images to generate thumbnails or metadata; denial of service is already relevant, and memory corruption can be more serious in a backend with access to stored media.

Our harnesses cover important libpng behavior, but they are not full application harnesses. Firefox uses progressive decoding: `nsPNGDecoder::InitInternal()` installs `info_callback`, `row_callback`, and `end_callback`, while `ReadPNGData()` feeds chunks incrementally. Our decoder instead calls the sequential `png_read_info()` / `png_read_image()` API on a complete input, missing browser-specific streaming states, callback ordering, eXIf parsing, qcms color transforms, and APNG frame paths. ImageMagick wraps libpng with blob I/O, resource limits, raw profile extraction via `Magick_png_read_raw_profile()`, MNG/JNG support, and application pixel-cache conversion; these glue paths can create state and lifetime patterns that isolated library harnesses intentionally simplify away.

[1] `https://www.libpng.org/pub/png/libpng.html`

[2] `https://searchfox.org/firefox-main/source/image/decoders/nsPNGDecoder.cpp`

[3] `https://github.com/ImageMagick/ImageMagick/blob/main/coders/png.c`

2.7 Q7 — Binary-Only Fuzzing with QEMU

The clean QEMU comparison uses the roughly equal 32-minute decoder runs. The speed row is confounded: the source-instrumented fork-mode run uses ASan, while the QEMU binary is an unsanitized `gcc` build. QEMU pays runtime translation overhead, but ASan's shadow-memory, redzone, and allocation checks are large enough here that the unsanitized QEMU run is slightly faster.

Run	Exec/s	Edges	Corpus
ASan fork	532.08	966 / 3,122	715
QEMU short	576.56	2,730 / 65,536	801

Table 3: Same-wall-clock source-instrumented and QEMU decoder campaigns.

The QEMU bitmap density looks lower, 4.17% versus 30.94%, because its denominator is AFL++'s 64K QEMU bitmap rather than the compile-time edge universe reported by `afl-clang-fast`. Therefore the raw densities and edge counts are not directly comparable as coverage percentages. The useful conclusion is qualitative: QEMU can fuzz a binary-only `gcc` build and found 2,730 bitmap entries in the same wall-clock window, but its coverage signal comes from runtime translation and should not be mixed directly with the instrumented ASan edge totals. The longer QEMU run lasted about 3 h 39 min and reached 2,917 bitmap entries with no crashes; it is supporting evidence, not a clean six-hour comparison.

2.8 Q8 — Instrumentation Depth and Performance

For instrumentation depth, we compared a minimal libpng probe to the final decoder harness using 30-second ASan-free AFL-instrumented runs. The probe is only a proxy for linked libpng instrumentation: it creates and destroys libpng state, but it is not a full parser campaign.

Binary	Reached	Total	Density
Minimal libpng probe	49	3,090	1.59%
Final decoder harness	808	3,130	25.81%

Table 4: Instrumented edge counts from `evidence/q8-edge-counts`.

The total edge counts are close because both binaries link the same instrumented libpng archive. The reached counts diverge because the harness feeds PNG bytes through `png_read_info()`, transform setup, row allocation, `png_read_image()`, and `png_read_end()`. Even the final harness reaches only 25.81% in this short run; the rest sits behind validity constraints, rare chunk orderings, transform combinations, zlib states, error handlers, and APIs the decoder harness does not call.

Configuration	Exec/s	Edges	Mode
No sanitizer + fork	1,612.57	793 / 3,130	fork
ASan + fork	684.78	752 / 3,129	fork
ASan + persistent	1,799.32	788 / 3,138	persistent

Table 5: `afl-fuzz -V 30` speed measurements with the same decoder logic.

These 30-second fixed-window measurements are still short, so the absolute speeds should not be over-read. The ordering captures the tradeoff. Removing ASan avoids shadow-memory checks and raises fork-mode speed from 684.78 to 1,612.57 exec/s. Persistent mode keeps ASan but loops in-process through `__AFL_LOOP`, removing most per-testcase process-startup cost; in this window it reached 1,799.32 exec/s, about 2.6 times ASan fork. The caveat is the same as in Q4: persistent-mode crashes need standalone replay before being called bugs. The persistent measurement also had slightly lower, but still high, stability at 99.87% with four variable corpus entries.

3 Conclusion

The required 30-minute decoder campaigns provide the clean baseline evidence for Q4 and Q7, while the longer campaigns show how additional budget and entry points affected coverage. The focused API harnesses expanded testing into public setter and writer interfaces, and the strongest confirmed finding is one text-API `png_set_text_2` root cause reproducing the CVE-2016-10087 bug class. The main limitations are also clear: decoder hangs remain pending, persistent crashes were excluded as non-reproducing, and bitmap/edge numbers must only be compared within the same instrumentation source.

A Evidence Artifacts

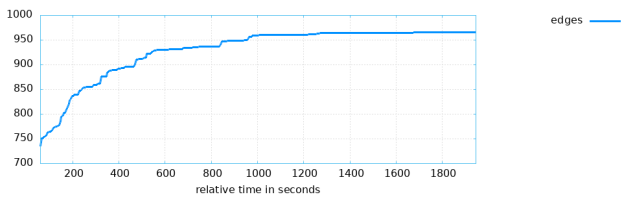

Figure 1: Baseline source-instrumented decoder edges over time.

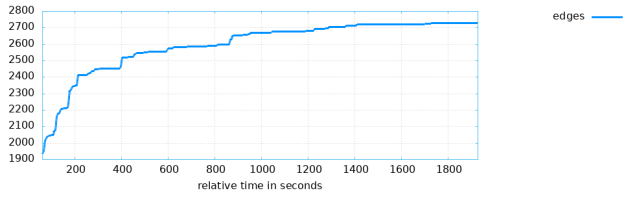

Figure 2: Baseline QEMU decoder edges over time.

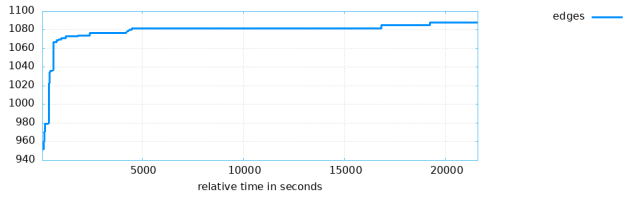

Figure 3: Long persistent decoder campaign edges over time.

Table 6 summarizes the longer and focused runs referenced from the body.

The main report numbers come from:

- Instrumented campaign: `findings/default/` and `plot_output/`.
- QEMU campaign: `findings-qemu/default/` and `plot_output_qemu/`.
- AFL++ status counters: `findings/default/fuzzer_stats` for Q4 and `findings-qemu/default/fuzzer_stats` for Q7.
- Long supporting campaigns: `triage-artifacts/` and `evidence/plots/`.
- Q3 strategy yields: `evidence/q3-live/`.
- Q5 triage: `pocs/text_api_triage/`.
- Q8 edge counts: `evidence/q8-edge-counts/`.
- Q8 speed measurements: `evidence/q8-speed/`.

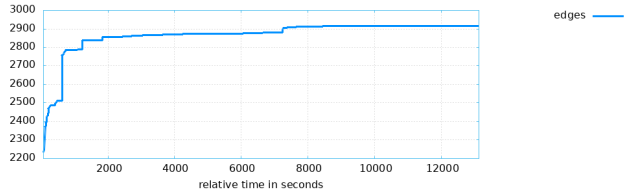

Figure 4: Long QEMU decoder campaign edges over time.

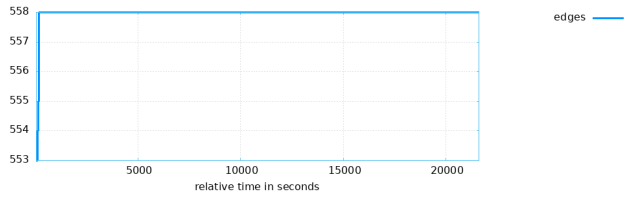

Figure 5: Long write-API campaign edges over time.

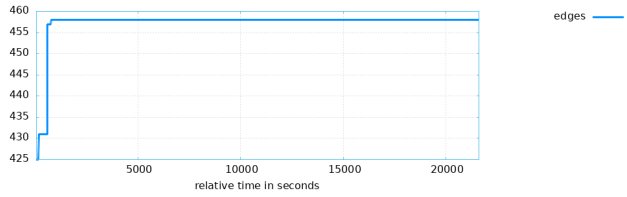

Figure 6: Long metadata-API campaign edges over time.

Supporting run	**Runtime**	**Edges**	**Outcome**
Persistent decoder	6.0 h	1,088 / 3,131	2 crash files, non-repro
Long QEMU decoder	3.6 h	2,917 / 65,536	0 crashes
Write API	6.0 h	558 / 2,090	0 crashes
Metadata API	6.0 h	458 / 3,388	0 crashes
Text API	5 min	241 / 3,227	90 saved, 59 repro

Table 6: Long and focused supporting campaign summary. Edge counts are from the main fuzzer instance for each campaign; text-API crash counts are from the deduplicated replay logs.